

Checklist

Secure Software Development

1. Language Selection

- ☐ Consider **security** when choosing a programming language.
- ☐ Evaluate the language's built-in protections against common vulnerabilities.
- ☐ Prefer memory-safe languages where practical.
- ☐ Understand the security risks associated with the selected language.
- ☐ Use security scanning tools and libraries available for the language.

Language-Specific Considerations

- ☐ **Assembly:** Use only when necessary; recognize the lack of safety checks and high risk of memory-related errors.
- ☐ **C/C++:** Carefully manage pointers and memory to prevent buffer overflows, use-after-free, double-free, and related vulnerabilities.
- ☐ **Go:** Take advantage of garbage collection, strong typing, bounds checking, and limited pointer arithmetic.
- ☐ **Go:** Avoid the **unsafe** package unless absolutely necessary.
- ☐ **Rust:** Use ownership and memory-safety protections to reduce memory-corruption vulnerabilities.
- ☐ **Python/Ruby/Perl:** Validate input carefully even though automatic memory management reduces many memory-related risks.
- ☐ **PHP:** Avoid insecure features and practices such as unsafe remote file inclusion and improperly validated user-controlled input.

2. Secure Coding Guidelines

- ☐ Establish documented secure coding standards for the development team.
- ☐ Apply the standards consistently across the codebase.

- ☐ Define **user-supplied input** broadly, including data from:
 - ☐ Networks
 - ☐ Files
 - ☐ Command-line input
 - ☐ GUI interactions
 - ☐ Peripheral devices
- ☐ Never process user input without validating it first.
- ☐ Confirm that input is the expected **data type**.
- ☐ Confirm that input is the expected **length**.
- ☐ Confirm that input falls within the expected **range**.
- ☐ Confirm that input is valid in relation to other data in the same transaction.
- ☐ Reject or discard invalid data appropriately.
- ☐ Use approved libraries or reusable code for common security-sensitive tasks.
- ☐ Establish standards for:
 - ☐ Cryptography
 - ☐ Database access
 - ☐ Memory management
 - ☐ Network communication
 - ☐ Error handling
 - ☐ Authentication
 - ☐ Audit logging
 - ☐ Access management
- ☐ For web applications, define secure session-management requirements.
- ☐ Define security requirements for communication between client-side and server-side components.

3. Security Testing

- ☐ Perform security testing before releasing code to production.
- ☐ Select testing methods appropriate to the application and development environment.
- ☐ Include security testing throughout development rather than only at release.

Automated Static Testing

- ☐ Scan source code without executing it.
- ☐ Integrate static analysis into the CI/CD pipeline when possible.
- ☐ Scan new and modified code whenever changes are committed.
- ☐ Review findings for false positives.

- ☐ Check for buffer overflows and other memory-related weaknesses.
- ☐ Check for use of unsafe or dangerous functions.
- ☐ Use secret-detection tools to identify accidentally committed:
 - ☐ Passwords
 - ☐ Tokens
 - ☐ API keys
 - ☐ Private cryptographic keys
- ☐ Supplement static analysis with other techniques because it may miss design-level vulnerabilities.

Automated Dynamic Testing

- ☐ Test the application while it is running.
- ☐ Supply a variety of inputs and observe application behavior.
- ☐ Test for vulnerabilities that may be difficult to detect statically.
- ☐ Test for improper input handling and injection vulnerabilities.
- ☐ Test for reflection or output-handling issues where relevant.
- ☐ Account for the possibility that dynamic testing may not provide complete coverage.

Peer Review

- ☐ Have another developer review new or modified code.
- ☐ Review code systematically for security weaknesses.
- ☐ Verify findings before requiring remediation.
- ☐ Use reviewers with appropriate knowledge of the language and vulnerability type.
- ☐ Combine peer review with automated testing rather than relying on it alone.

4. Secure Software Development Lifecycle

Stage 1: Training

- ☐ Train developers in secure software-development practices.
- ☐ Provide training in threat modeling.
- ☐ Provide training in security testing.
- ☐ Provide relevant privacy training.
- ☐ Ensure developers understand both how to write secure code and how to interpret security-testing results.

Stage 2: Requirements

- ☐ Identify security requirements at the same time as functional requirements.
- ☐ Treat security requirements as mandatory requirements rather than optional additions.
- ☐ Document security requirements clearly.
- ☐ Keep security requirements visible throughout the project.

Stage 3: Architecture and Design

- ☐ Define security expectations before coding begins.
- ☐ Identify where encryption is required.
- ☐ Identify where access controls are required.
- ☐ Identify how sensitive information will be stored and handled.
- ☐ Perform threat modeling during architecture and design.
- ☐ Identify high-level attack methods.
- ☐ Review proposed designs for weaknesses.
- ☐ Modify the design to reduce the attack surface.
- ☐ Ensure security controls are implemented in the appropriate locations.

Stage 4: Code and Build

- ☐ Develop according to approved architecture and design specifications.
- ☐ Use the predetermined programming language.
- ☐ Follow organizational secure coding guidelines.
- ☐ Apply agreed-upon security best practices.
- ☐ Use approved libraries and development tools.

Stage 5: Test and Review

- ☐ Test functionality.
- ☐ Perform security testing.
- ☐ Use multiple security-testing methods where appropriate.
- ☐ Identify and document security defects.
- ☐ Assess the severity and impact of defects.
- ☐ Return vulnerable code to developers for remediation.
- ☐ Retest security fixes before release.

Stage 6: Release

- ☐ Follow the organization's formal release process.
- ☐ Conduct a final security review.
- ☐ Confirm that identified security defects have been addressed.
- ☐ Prepare required operational documentation.
- ☐ Ensure incident-response procedures are in place.
- ☐ Ensure business-continuity procedures are in place.
- ☐ Transfer appropriate information and responsibilities to operational support teams.

Final Review

- ☐ Secure coding standards are documented and followed.
- ☐ User input is validated before processing.
- ☐ Appropriate static, dynamic, and manual testing has been completed.
- ☐ Security has been incorporated into every SDLC stage.
- ☐ Known vulnerabilities have been remediated or formally addressed.
- ☐ The application is ready for a final security review before production release.